PÓS-GRADUAÇÃO EM ARQUITETURA DE SOFTWARE · FIAP (SOAT)

Tech Challenge — Fase 4

Sistema de Gestão de Oficina Mecânica — Arquitetura de Microsserviços, Persistência Poliglota e Saga Pattern

1. Identificação do Aluno

Integrante	**Vladmir Musa Costa Bravo**
RM	**374193**
Discord	`@vladbravo`
Curso / Fase	Pós-graduação em Arquitetura de Software — FIAP (SOAT) · Entrega Fase 4

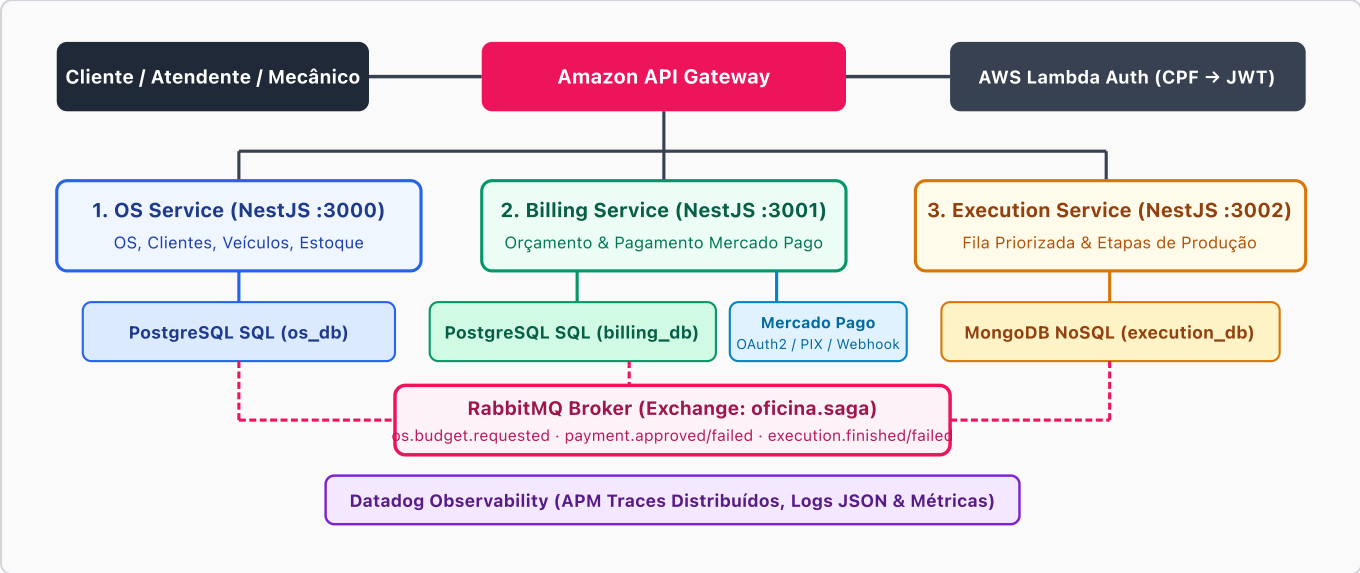
2. Links dos Repositórios GitHub e Vídeo de Demonstração

A solução foi decomposta em **3 microsserviços independentes**, cada um com seu próprio repositório, infraestrutura Terraform (`/infra`), manifestos Kubernetes (`/k8s`), banco de dados isolado (Database per Service), pipeline de CI/CD com validação de testes/qualidade (SonarQube), branch `main` protegida por Pull Request e colaborador `soat-architecture` :

Componente / Microsserviço	Banco de Dados	Infra & K8s	Repositório GitHub
1. OS Service (Ordem de Serviço, Clientes, Veículos, Estoque)	**PostgreSQL (SQL)** (`os_db`)	`/infra` + `/k8s`	https://github.com/VladMCBravo/tech-challenge-fase4-os-service
2. Billing Service (Orçamento, OAuth2 e Pagamento Mercado Pago)	**PostgreSQL (SQL)** (`billing_db`)	`/infra` + `/k8s`	https://github.com/VladMCBravo/tech-challenge-fase4-billing-service
3. Execution Service (Fila de Execução e Produção)	**MongoDB (NoSQL)** (`execution_db`)	`/infra` + `/k8s`	https://github.com/VladMCBravo/tech-challenge-fase4-execution-service
4. Function Serverless (Autenticação CPF → JWT)	—	Terraform	https://github.com/VladMCBravo/tech-challenge-fase3-serverless-auth
5. Infraestrutura Kubernetes & API Gateway	—	Terraform	https://github.com/VladMCBravo/tech-challenge-fase3-infra-k8s
6. Infraestrutura Banco de Dados	—	Terraform	https://github.com/VladMCBravo/tech-challenge-fase3-infra-db

> **Link do Vídeo de Demonstração (YouTube — até 15 min):** `<INSERIR_LINK_DO_YOUTUBE_AQUI>`
> **Documentação Completa e Roteiro no Repositório:** `docs/Entrega-Fase4-TechChallenge.md` e `docs/Roteiro-Video-Fase4.md`

3. Diagrama Geral da Arquitetura Final



4. Justificativa da Divisão dos Microsserviços e Persistência Poliglota (SQL + NoSQL)

1. **`os-service` (Ordem de Serviço + PostgreSQL SQL — `os_db`):** Concentra o domínio cadastral, o ciclo de vida da OS e o controle de estoque. Utiliza banco relacional **SQL (PostgreSQL)** para garantir integridade referencial estrita e transações ACID locais na reserva preventiva (`DECREASE`) e no estorno (`REVERSAL`) de peças.
2. **`billing-service` (Orçamento e Pagamento + PostgreSQL SQL — `billing_db` + Mercado Pago):** Isola o contexto financeiro, a autenticação OAuth2 via `client_id` e `client_secret` (`POST /api/budgets/mercadopago/oauth/token`), a geração de preferências Checkout Pro / QR Code PIX no **Mercado Pago** (com fallback Mock resiliente quando credenciais externas de teste estão indisponíveis) e o processamento de Webhooks. Utiliza **SQL (PostgreSQL)** pela exigência de precisão decimal exata (`Decimal(10,2)`) e auditoria financeira.
3. **`execution-service` (Execução e Produção + MongoDB NoSQL — `execution_db`):** Gerencia a fila priorizada de pátio (`IN_REPAIR > IN_DIAGNOSIS > QUEUED`) e os apontamentos de diagnóstico/reparo. Utiliza **NoSQL (MongoDB)** porque cada execução é um documento agregado (`execution_tasks`) contendo os itens da OS e o histórico dinâmico de etapas (`steps[]`), proporcionando leituras e escritas rápidas sem necessidade de `JOINs` .

5. Estratégia Escolhida para o Saga Pattern (Coreografia via RabbitMQ)

Foi adotado o **Saga Pattern Coreografado (Event-Driven Choreography)** via **RabbitMQ** (`exchange: oficina.saga` , tipo `topic`), garantindo baixo acoplamento e ausência de ponto único de falha:

- **Fluxo de Sucesso (Happy Path):**
  1. **Reserva e Solicitação de Orçamento (`os-service`):** Ao mover a OS para `WAITING_APPROVAL` , reserva o estoque das peças (`DECREASE`) e publica `os.budget.requested` .
  2. **Geração de Orçamento e Cobrança (`billing-service`):** Consome `os.budget.requested` , obtém o token OAuth2 via `client_id` / `client_secret` e gera o orçamento (`PENDING`) e o link/QR Code PIX no Mercado Pago.
  3. **Aprovação do Pagamento (`billing-service` → `os-service` + `execution-service`):** O Webhook do Mercado Pago confirma o pagamento e publica `payment.approved` . O `os-service` muda a OS para `IN_EXECUTION` e publica `os.execution.requested` ; o `execution-service` insere a OS na fila NoSQL do MongoDB (`QUEUED`).

4. **Conclusão do Reparo (`execution-service` → `os-service`):** Ao concluir o serviço no MongoDB (`COMPLETED`), publica `execution.finished` e o `os-service` atualiza a OS para `FINISHED` .

- **Fluxos de Compensação e Rollback (Tratamento de Falhas):**
  - **Cenário 1 — Pagamento Recusado no Mercado Pago (`payment.failed`):** O `billing-service` marca o pagamento como `REJECTED` e publica `payment.failed` . O `os-service` consome o evento, executa a **transação compensatória devolvendo as peças ao estoque (`REVERSAL`)** e altera a OS para `CANCELLED` .
  - **Cenário 2 — Falha Crítica na Execução (`execution.failed`):** O `execution-service` marca a tarefa como `FAILED` no MongoDB e publica `execution.failed` . O `billing-service` consome o evento e **estorna o pagamento no Mercado Pago (`REFUNDED`)**, enquanto o `os-service` **devolve as peças ao estoque (`REVERSAL`)** e cancela a OS (`CANCELLED`).

6. Qualidade, Testes Unitários (> 80%), BDD, CI/CD e Observabilidade

Microsserviço	Suítes / Testes	Cobertura (Lines)	BDD / SonarQube / Deploy Automatizado K8s
`os-service`	11 suítes / 117 testes	**90,50%**	Cenários BDD em Gherkin (`work-order-saga.feature`) + SonarQube + Deploy K8s
`billing-service`	1 suíte / 10 testes	**83,68%**	OAuth2 (`client_id` / `client_secret`) + Checkout/PIX Mercado Pago + Rollback `REFUNDED` + SonarQube + Deploy K8s
`execution-service`	1 suíte / 7 testes	**81,40%**	Fila NoSQL MongoDB + Eventos Saga + Rollback + SonarQube + Deploy K8s

> **Nota sobre Deploy Automatizado em Kubernetes e Limitação do AWS Academy:** Conforme orientação dos professores e coordenação, os pipelines `.github/workflows/deploy.yml` dos 3 repositórios possuem gatilhos automáticos (`push` , `pull_request`) e `workflow_dispatch` . Em cada execução, o job `deploy-k8s` provisiona automaticamente um cluster Kubernetes via **Kind (`helm/kind-action`)** no próprio pipeline, aplica todos os manifestos de `/k8s` e valida o status dos Pods, Services, HPA e Health Checks, além de realizar o deploy no **Amazon EKS** quando as credenciais temporárias do AWS Academy estão ativas.